Department of Computer Engineering

CSE5041 Database Design & Development
Project Report

DIY Skill-Sharing & Mentorship App Database System

Students :

ID	Name & Surname
2300005352	Barış Bağbekleyen
2300002912	Mert Eser

TABLE OF CONTENTS

1. INTRODUCTION 4
2 ENTITY RELATIONAL MODEL 5
2.1 ENHANCED ER DIAGRAM 5
2.2 RELATIONAL SCHEMA & MAPPING 6
3 NORMALIZATION 7
3.1 FUNCTIONAL DEPENDENCIES 7
3.2 UNNORMALISED FORM 8
3.3 FIRST NORMAL FORM 8
3.4 SECOND NORMAL FORM 9
3.5 THIRD NORMAL FORM 9

SAMPLE

LIST OF FIGURES

Figure 1: EER diagram of the Company Employee Administrative Database 5
Figure 2: Relational schema of the Company Employee Administrative Database with arrows indicating referential integrity..... 6

SAMPLE

1. INTRODUCTION

1.1 PROJECT DESCRIPTION

This project involves the design, development, and implementation of a relational database system for a "DIY Skill-Sharing Application." The platform serves as a marketplace for knowledge, connecting individuals who possess specialized skills (Mentors) with those eager to learn new hobbies (Learners) such as woodworking, coding, pottery, and mechanics.

The core functionality of the system includes:

- **Advanced User Management:** Distinguishing between Mentors and Learners while maintaining a centralized user identity.
- **Course & Workshop Management:** Handling scheduling, pricing, capacity limits, and location details.
- **Dynamic Enrollment System:** Managing registrations, automated waitlists for full classes, and preventing scheduling conflicts.
- **Financial & Performance Analysis:** Calculating revenue per workshop/mentor and analyzing student performance via transcripts.
- **Feedback & Gamification:** A robust review system and a badge-based achievement system to motivate learners.

The system is designed to handle complex business logic—such as age verification and prerequisites—directly within the database layer using Stored Procedures and Triggers, ensuring data integrity and high performance.

SAMPLE

2 ENTITY RELATIONAL MODEL

2.1 ENHANCED ER DIAGRAM

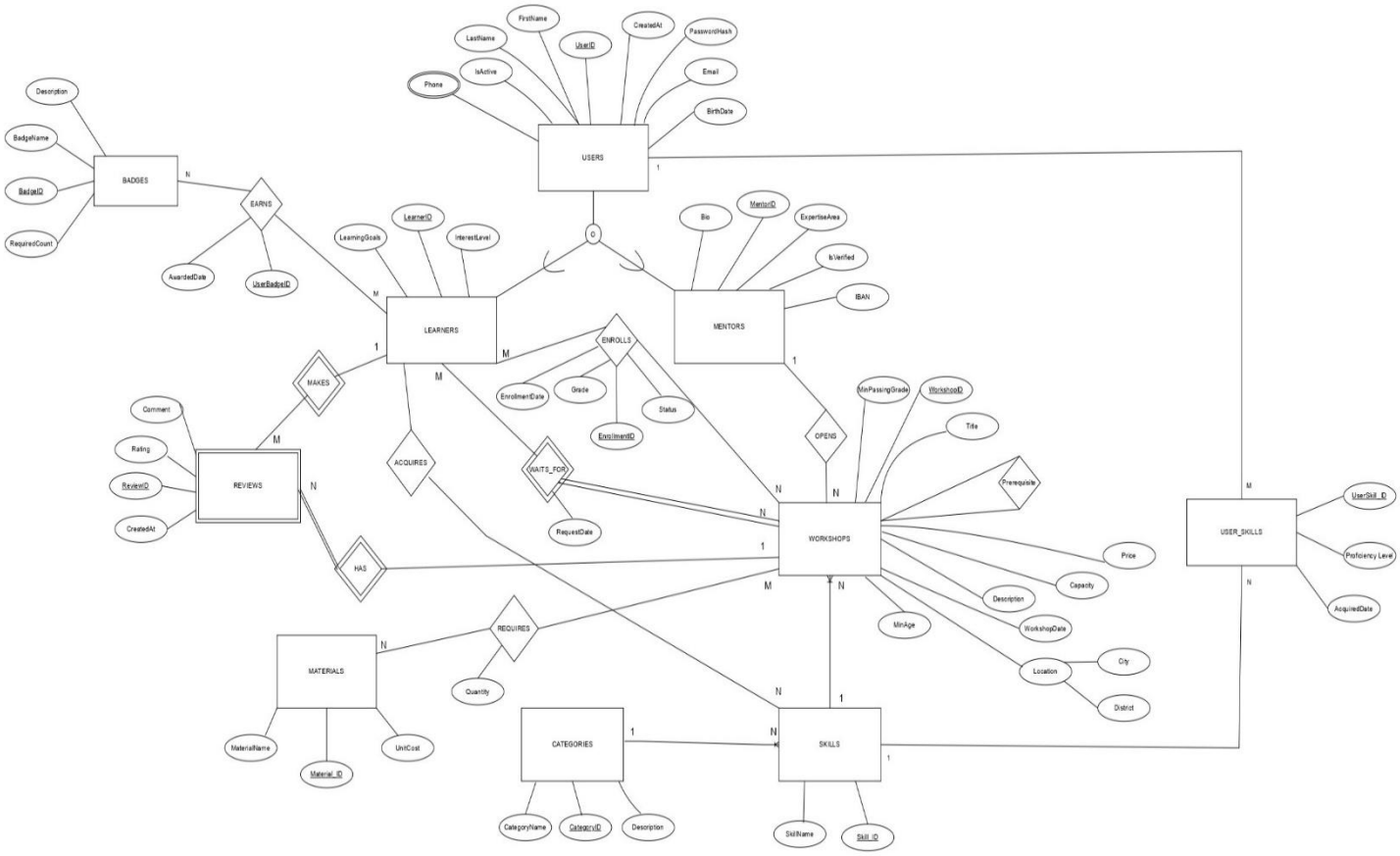

Figure 1: EER diagram of the Company Employee Administrative Database

2.2 RELATIONAL SCHEMA & MAPPING

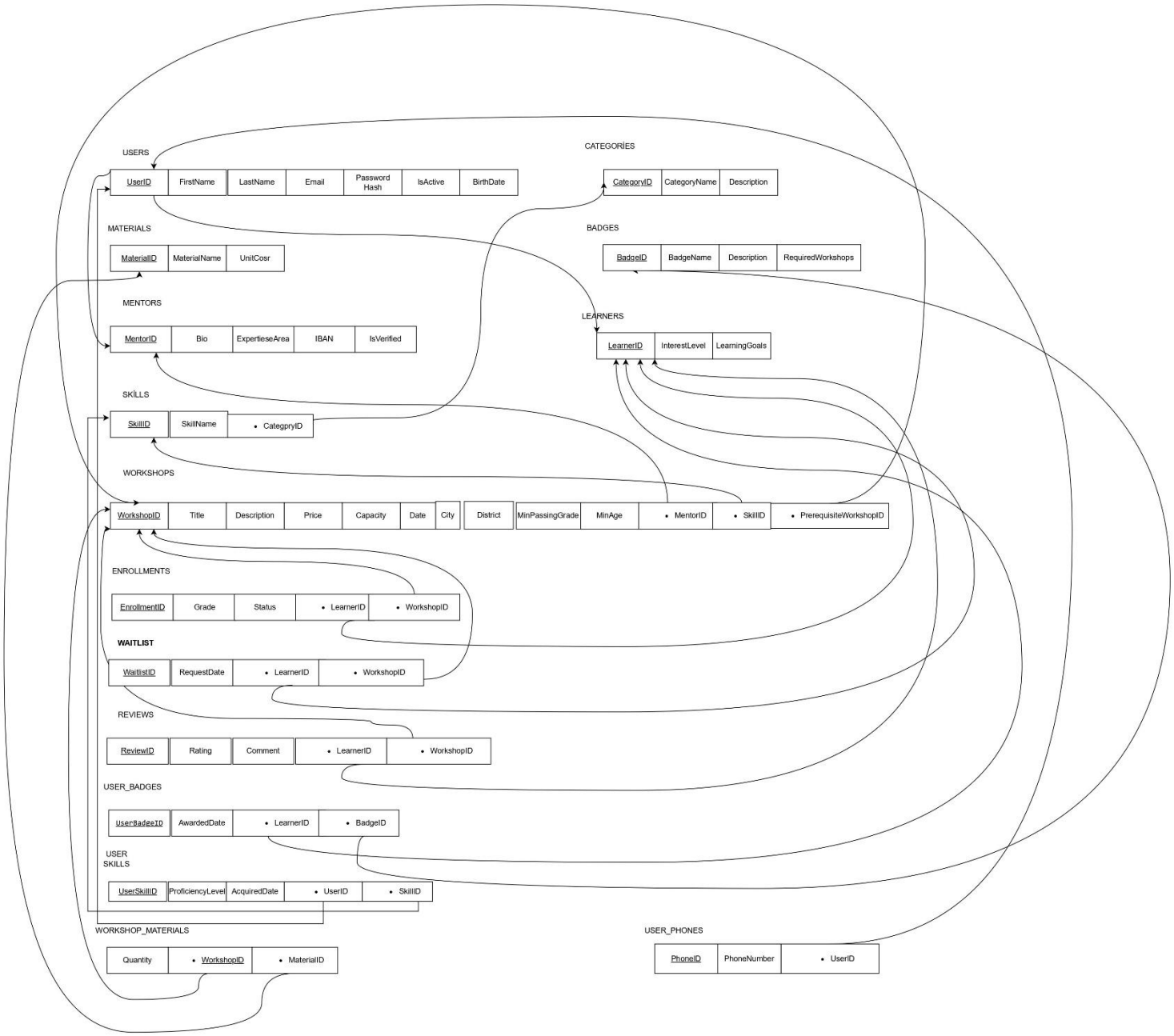

Figure 2: Relational schema of the Company Employee Administrative Database with arrows indicating referential integrity

3 NORMALIZATION

3.1 FUNCTIONAL DEPENDENCIES

To ensure data consistency, the following dependencies were identified and enforced:

FD1 (Users): UserID {FirstName, LastName, Email, PasswordHash, IsActive, BirthDate, CreatedAt}

FD2 (Mentors): MentorID {Bio, ExpertiseArea, IBAN, IsVerified}

FD3 (Learners): LearnerID {InterestLevel, LearningGoals}

FD4 (UserPhones): PhoneID {UserID, PhoneNumber, PhoneType}

FD5 (Categories): CategoryID {CategoryName, Description}

FD6 (Skills): SkillID {SkillName, CategoryID}

FD7 (UserSkills): UserSkillID {UserID, SkillID, ProficiencyLevel, AcquiredDate}

FD8 (Workshops): WorkshopID {Title, Description, Price, Capacity, WorkshopDate, Location, City, District, MentorID, SkillID, MinPassingGrade, MinAge, PrerequisiteWorkshopID}

FD9 (Enrollments): EnrollmentID {WorkshopID, LearnerID, EnrollmentDate, Grade, Status}

FD10 (Materials): MaterialID {MaterialName, UnitCost}

FD11 (WorkshopMaterials): {WorkshopID, MaterialID} {Quantity}

FD12 (Badges): BadgeID {BadgeName, Description, RequiredWorkshops}

FD13 (UserBadges): UserBadgeID {LearnerID, BadgeID, AwardedDate}

SAMPLE

FD14 (Reviews): ReviewID {WorkshopID, LearnerID, Rating, Comment, CreatedAt}

FD15 (Waitlist): WaitlistID {WorkshopID, LearnerID, RequestDate}

3.2 UNNORMALISED FORM

At the initial stage of our design, we imagined the entire database as a single, massive list to identify potential problems. In this unnormalized state, we faced a chaotic structure where unrelated data was mixed together.

The Problem was, a single row in this hypothetical list would try to store everything: the student's name, the workshop they are attending, the mentor's bank details, and the list of materials needed. This meant that if a workshop had 20 students, we would have to repeat the mentor's IBAN and biography 20 times, creating a huge amount of redundant data.

3.3 FIRST NORMAL FORM

To bring order to this structure (1NF), we focused on breaking down composite values and separating repeating groups.

Isolating Contact Info: We noticed that users might have multiple phone numbers (e.g., one for work, one for personal use). Storing these in a single cell (e.g., "0555... , 0532...") would make queries impossible. Solution: We moved all phone numbers to a dedicated USER_PHONES table, linking them back to the user.

Splitting Locations: Instead of storing a workshop's location as a long address string, we split it into City and District columns. This allows us to easily filter workshops by region in the future.

Defining Keys: We assigned a unique ID (Primary Key) to every major entity, ensuring we can identify any single record without confusion.

Handling Multiple Skills: Similarly, a user acts as both a learner and a mentor, possessing multiple skills ('Python', 'Woodworking', 'Cooking'). In a flawed design, we might have listed these as a comma-separated string in the Users table. To adhere to 1NF, we strictly avoided this by creating the USER_SKILLS table. This ensures that every single skill proficiency is stored as an atomic record, allowing us to query 'Who knows Python?' efficiently without parsing text strings.

SAMPLE

Result: We eliminated the "messy cell" problem, but we still had data repeating in rows (like a mentor's expertise area repeating for every class they teach).

3.4 SECOND NORMAL FORM

In this step, we examined our tables to ensure every piece of data depended on the whole primary key, not just a part of it. This was particularly important for our "Many-to-Many" relationships.

The Critical Check: We looked closely at the WORKSHOP_MATERIALS table, which uses a composite key (WorkshopID + MaterialID).

Our Observation: The column Quantity (how many items are needed) doesn't depend solely on the Workshop (because different workshops need different materials) nor solely on the Material (because the same material might be used in different amounts). It depends on the specific combination of both.

Analysis of User Proficiency: Another clear application of 2NF is in our USER_SKILLS table. This table conceptually links UserID and SkillID. The attribute ProficiencyLevel (e.g., Beginner, Advanced) relies on the specific combination of a user and a skill.

- It does not depend only on the User (a user has different levels for different skills).

- It does not depend only on the Skill (different users have different proficiencies). Since the proficiency level depends on the intersection of both keys, it confirms our compliance with 2NF."

Conclusion: Since our data relies on the full key combination, and our single-key tables (like Users) are naturally compliant, our schema successfully meets the 2NF standards.

3.5 THIRD NORMAL FORM

Finally, we eliminated "hidden" dependencies where data relied on non-key attributes (Transitive Dependencies). We essentially asked: "Does this data really belong to this primary key?"

SAMPLE

Refining Skills: Initially, a skill record might have included the category name. However, the category name depends on the CategoryID, not the skill itself. To fix this, we moved category details to a separate CATEGORIES table.

Mentor Details: Storing a mentor's IBAN or Bio inside the WORKSHOPS table creates redundancy. If a mentor changed their IBAN, we would have to update every workshop record they taught.

- **Solution**: We kept only the MentorID in the workshop table and moved all personal and financial details to the MENTORS table.

Review System Integrity: In the REVIEWS table, we store the Rating and Comment associated with a ReviewID. A common mistake would be to include the WorkshopTitle or LearnerName here for readability. However, WorkshopTitle depends on WorkshopID, and LearnerName depends on LearnerID. Including them would create a Transitive Dependency. By storing only the Foreign Keys (WorkshopID, LearnerID), we eliminated redundancy.

Final Design: By systematically applying these three steps, we transformed a flat, repetitive list into a streamlined relational schema where every piece of data lives in exactly one logical place.

SAMPLE